TCSS 343 – Design and Analysis of Algorithms

Job Selection Revisited

Job Selection (Revisited)

- Let's consider another variant of the Job Selection problem:

  Job Selection Revisited (JOB2)
  Input: $P[1 \dots n]$ a list of n positive numbers and $0 \leq k \leq n$ and integer
  Output: m a maximum possible payout and $A \subseteq \{1, \dots, n\}$ a set of indexes such that $|A| = k$ and such that $m = \sum_{i \in A} P[i]$

- In this variant we don't have a constraint about which jobs we can work. Rather, we have a constraint about how many jobs we can work.
  - Changing the constraint has made the problem easier to solve because we can deploy a simple greedy algorithm.
  - What jobs should we select to maximize profit?

- To maximize total profit, we should select the k jobs with maximum payout.
  - Are you sure this is correct?
  - Can you prove it correct?

Greedy Algorithms

- A problem has the greedy choice property if making a locally optimal choice leads to a globally optimal solution to the problem.
  - Making a greedy choice is another means of reducing a problem to a sub-problem.
  - Unlike Dynamic Programming the greedy choices will not be reconsidered in a greedy algorithm.
  - Greedy_algorithm

- There is often more than one greedy choice we might make locally.
  - Some of these choices will have the greedy choice property whereas others will not.
  - Counter Example: To prove the greedy choice is incorrect we need only find an instance of the problem that it does not work on.
  - Proof of Correctness: To prove the greedy choice correct we use a form of induction.

Proof of Correctness

Theorem: Selecting the k jobs with maximum payout will maximize total payout.

Proof: Consider a set of the k jobs with maximum payout and call this set S. Then, $T = \sum_{s \in S} s$ is the total payout of this set.

Imagine replacing a job $s \in S$ with any other job $s' \notin S$. Then, $T' = \sum_{s \in S} s \ - s + s'$ is the total payout of the new set.

$$T - T' = \sum_{s \in S} s - \left(\sum_{s \in S} s - s + s' \right)$$

$$= \sum_{s \in S} s - \sum_{s \in S} s + s - s'$$

$$= s - s'$$

Since s was among the k jobs with maximum payout and s' was not we know that $s \geq s'$.

So $s - s' \geq 0$, and thus $T - T' \geq 0$, implying that $T \geq T'$.

So, the total payout of S is greater than or equal to the total payout of any other set of jobs. □

Proof of Correctness

- A proof of correctness must prove for every input that the algorithm produces the correct output.
  - Generality: It must work for any input, so we cannot assume the input is of a particular form.
    - Risk: Proof by example (aka inappropriate generalization).
  - Universal: We must show our solution is better (or equal to) all possible alternatives.
    - Risk: We forget critical cases and risk proof by example.
  - Complete: We must include all relevant information in our arguments.
    - Risk: We leave our arguments vague and hard to understand and may miss errors in our reasoning or beg the question.
  - Precise: We must use terms as they are defined and avoid equivocation.
    - Risk: Imprecise language can hide errors in our reasoning.
  - Concise: We don't want to include information that is not necessary.
    - Risk: We can confuse our reader by stating something false or irrelevant.

- These constraints on proof writing make it an art, and every proof should consider its intended audience.
  - For proofs in this course, I recommend considering your fellow university students as the intended audience.
  - In the last proof I prioritized a concise proof over a complete proof, and so many parts have been left out.

Proof By Example (Fallacy)

- Another tempting fallacy to commit is called “proof by example” or “inappropriate generalization”.
- Proof by example – a proof where we attempt to prove a generalization with a specific instance.
- Example: “I know a corrupt politician, so all politicians must be corrupt.”
  - It’s possible that not all politicians are corrupt.
  - All we can infer is that some politicians are corrupt (at least one).
- Why is this bad?
  - We can prove things that are false, from a single instance that is true.
    - The sum of any two numbers is 7, since $4 + 3 = 7$.
    - All integers are 0, since 0 is an integer.
  - This is an error in applying a universal quantifier when we can only infer an existential quantifier.

Begging the Question (Fallacy)

- In many proofs for greedy algorithms, you will be tempted to commit a fallacy called “begging the question”.
  - A fallacy is a logical error or error in reasoning.
  - When we commit a fallacy, we might end up with falsehoods.

- Begging the question – A proof in which you assume what you are trying to proof.

- Example: “Everyone likes Tom Hanks, because he is just so likable.”
  - We could rephrase this as “Everyone likes Tom Hanks because everyone likes Tom Hanks.” which does not add any information or help us understand why everyone likes Tom Hanks.

- Why is this bad?
  - We can prove anything we want this way:
    - $1 + 1 = 3$ because $3 = 1 + 1$.
    - $1 + 1 = 2$ because $2 = 1 + 1$.
  - We can’t tell which of these proofs tells us the truth, because they both beg the question. So, we should trust either of them (even though the second one is, in fact, true).

Theorem: Selecting the k jobs with maximum payout will maximize total payout.
Proof (begging the question): If we select any other jobs the total sum will not be greater. □

- While the statement is indeed true, this fails to prove the theorem, since we have not added any information for our audience.

Proof of Correctness

Theorem: Selecting the k jobs with maximum payout will maximize total payout.

Proof: Assume that the input consists of n jobs with arbitrary payouts and $0 \leq k \leq n$.
Let S be the set of k jobs with maximum payout.
Let O be a set of k jobs that has maximum total payout.
Assume S and O are in decreasing order of payout.
If $S = O$ we know that S has maximum total payout.
Assume $S \neq O$ and compare the jobs selected by S and O one at a time. Let i be the first job where S selects s_i and O selects $o_i \neq s_i$.
Since S is the set of k jobs with maximum payout we know that $s_i \geq o_i$.
Since O is in decreasing order of payout we know that s_i cannot appear later in O, so it is not selected by O.
Consider removing o_i from O and adding s_i and call the new set O'.
After this replacement $|O'| = k$, so O' is still a valid solution.
Then, $T = \sum_{o \in O} o$ is the total payout of O and $T' = \sum_{o \in O'} o$ is the total payout of O'.

$$T - T' = \sum_{o \in O} o - \left(\sum_{o \in O'} o \right)$$
$$= o_i - s_i$$

Since $s_i \geq o_i$ we know that $o_i - s_i \leq 0$, and thus $T - T' \leq 0$, implying that $T \leq T'$. This means that the O' has a greater total payout which is a contradiction to our assumption that O has the maximum total payout. As a result of this contradiction our assumption that $S \neq O$ must be false.
So, S, the set of k jobs with maximum payout also has maximum total payout. □

Proof of Correctness

Theorem: Selecting the k jobs with maximum payout will maximize total payout.

Proof (by induction on k): Assume that the input consists of n jobs with arbitrary payouts and $0 \leq k \leq n$.

Base Case $(k = 0)$: If we must select no jobs, then the maximum payout is 0 by selecting the empty set of jobs.

Inductive Hypothesis: Assume that selecting the k jobs with maximum payout will maximize total payout.

Inductive Case $(k \geq 0)$: We will show that selecting the $k + 1$ jobs with maximum payout will maximize total payout.
The set of $k + 1$ jobs with maximum payout contain the set, S, of k jobs with maximum payout. We know that S maximizes total payout over all other sets of k jobs (by the inductive hypothesis). So, to consider all possible sets of size $k + 1$ it is sufficient to consider the sets created by adding a single job to S.
Of the jobs not in S call the one with maximum payout s. So, the set of $k + 1$ jobs with maximum payout is $S \cup \{s\}$. Consider another set of $k + 1$ jobs, $S \cup \{o\}$, where $o \neq s$ is another job not in S.
Then, $T = \sum_{a \in S \cup \{s\}} a$ is the total payout of the set of $k + 1$ jobs with maximum payout and $T' = \sum_{a \in S \cup \{o\}} a$ is the total payout of our other set.

$$T - T' = \sum_{a \in S} a + s - \left(\sum_{a \in S} a + o \right)$$

$$= \sum_{a \in S} a - \sum_{a \in S} a + s - o$$

$$= s - o$$

Since s was the job not in S with maximum payout and o is any other job not in S we know that $s \geq o$.
So $s - o \geq 0$, and thus $T - T' \geq 0$, implying that $T \geq T'$.
So, the total payout of $S \cup \{s\}$ is greater than or equal to the total payout of any other set of $k + 1$ jobs.

Therefore, by induction selecting the k jobs with maximum payout will maximum total payout. □

Proof of Correctness

Theorem: Selecting the k jobs with maximum payout will maximize total payout.

Proof: Consider a set of the k jobs with maximum payout and call this set S. Then, $T = \sum_{s \in S} s$ is the total payout of this set.

Imagine replacing a job $s \in S$ with any other job $s' \notin S$. Then, $T' = \sum_{s \in S} s \ - s + s'$ is the total payout of the new set.

$$T - T' = \sum_{s \in S} s - \left(\sum_{s \in S} s - s + s' \right)$$

$$= \sum_{s \in S} s - \sum_{s \in S} s + s - s'$$

$$= s - s'$$

Since s was among the k jobs with maximum payout and s' was not we know that $s \geq s'$.

So $s - s' \geq 0$, and thus $T - T' \geq 0$, implying that $T \geq T'$.

So, the total payout of S is greater than or equal to the total payout of any other set of jobs. □

TCSS 343 – Design and Analysis of Algorithms

Library Books

Library Books

- In order to get the best grade in Algorithms you have taken out n books from the library. Unfortunately, you did not read any of them and they are now all overdue. You will be charged \$1/day per book. You have estimates of how long each book will take to read (in days) and have decided to read the books one at a time, returning the book once it's completed. You want to determine the order you should read the books in order to minimize your overdue costs.

- Can we formalize this problem?

```
Library Books
Input: T[1 ... n] a list of reading times for n books
Output: T', a permutation of T, the order the books should be read in to achieve the minimum cost m
```

- Consider an instance of this problem.

$$T = [5,2,4,1,3,6]$$

- What is the cost of reading the books in the order they are listed above?

$$\begin{aligned} &6 \cdot 5 + 5 \cdot 2 + 4 \cdot 4 + 3 \cdot 1 + 2 \cdot 3 + 1 \cdot 6 \\ &= 30 + 10 + 16 + 3 + 6 + 6 \\ &= \$71 \end{aligned}$$

- How can we express the total overdue cost in terms of a solution T'?

$$\sum_{i=1}^{n} (n - i + 1) T'[i]$$

Library Books

```
Library Books
Input: T[1 … n] a list of reading times for n books
Output: T′, a permutation of T, the order the books should be read in to achieve the minimum cost m
```

- Consider an instance of this problem.

$$T = [5,2,4,1,3,6]$$

- Consider solutions to this instance.
  - A solution is some ordering of the books.

$$S_0 = [5,2,4,1,3,6]$$
$$S_1 = [6,5,4,3,2,1]$$
$$S_2 = [4,3,2,1]$$
$$S_3 = [6,6,6,6,6,6]$$
$$S_4 = [1,2,3,4,5,6]$$

- A solution is valid if it is an acceptable solution to the problem (though possibly not optimal).
  - All the above solutions are valid except S_2 which does not list all the books and S_3 which reads the same book over and over.

- A solution is optimal if it is a valid solution that also achieves the optimum.
  - Of the valid solutions above only S_4 is optimal. How can we prove this?

Library Books

Library Books
Input: $T[1 \ldots n]$ a list of reading times for n books
Output: T', a permutation of T, the order the books should be read in to achieve the minimum cost m

- Notice that a greedy algorithm is often so simple we will describe it in a sentence or two instead of pseudocode.

```
LibraryBooks(T[1 ... n])
        Sort T in ascending order of reading time.
Return (T)
```

- We can express this algorithm simply in several different ways using natural language:
  - Read books in ascending order of length.
  - Always read the shortest book first.
  - Read the books from shortest to longest.

- What is the worst-case number of operations carried out by this algorithm?
  - We need to sort T. This will take $\Theta(n \log n)$.

- We still need to prove this algorithm correct.

Library Books

Theorem: Always reading and returning the shortest book first will minimize the overdue cost.

Proof: Assume we have an arbitrary list of reading times $T[1 \ldots n]$.

Let S be the list of books in increasing order of reading time. This is the order we will read and return the books.

Let O be the list of books in an order that minimizes the overdue cost. This is an optimal solution.

Now consider comparing S to O one book at a time starting with the first book. If S and O read the same book the we know we are making an optimal choice. So let's assume at some point S and O read different books and let s_i and o_i refer to the reading times of the two books. This gives us three possible cases.

Case 1: $o_i < s_i$

In this case O has selected a shorter book than we have. This contradicts that we always select the shortest book.

Case 2: $o_i = s_i$

In this case we have two books with the same reading time. We can alter O by swapping our book with the book selected by O. Call the new solution O'. We note that after the swap O' is still valid (it reads all the books). Since the reading time of both books are equal the cost of O' remains the same so O' is still optimal. We now have an optimal solution that reads the same book that we selected. We continue the proof with O' in place of O.

Case 3: $o_i > s_i$

In this case O has selected a longer book than we have. Once again, we can alter O by swapping our book with the book selected by O. Call the new solution O'. Let $c(O)$ express the cost of O. Then

$$c(O) = \sum_{k=1}^{n} (n - k + 1) o_k$$

and

$$c(O') = \sum_{k=1}^{n} (n - k + 1) o'_k$$

Library Books

Theorem: Always reading and returning the shortest book first will minimize the overdue cost.

Proof: (continued)

Assume that O reads our book as the jth book where $j > i$. Then

$$c(O) - c(O') = \sum_{k=1}^{n}(n-k+1)o_k - \sum_{k=1}^{n}(n-k+1)o'_k$$

$$= (n-i+1)o_i + (n-j+1)s_i - (n-i+1)s_i - (n-j+1)o_i$$

$$= (n-i+1)(o_i - s_i) + (n-j+1)(s_i - o_i)$$

$$= (n-i+1)(o_i - s_i) - (n-j+1)(o_i - s_i)$$

$$= \big((n-i+1) - (n-j+1)\big)(o_i - s_i)$$

$$= (j-i)(o_i - s_i)$$

$$> 0$$

since $j > i$ and $o_i > s_i$.

Since the difference in cost is positive this means $c(O) > c(O')$ and contradicts the assumption that O was optimal.

In cases 1 and 3 we have reached contradictions, so these cases are impossible. In case 2 after modifying O we have a valid and optimal solution that reads the same book as we do. Thus reading and returning the shortest book first will minimize the overdue cost. □

Proof of Correctness

Theorem: Our greedy algorithm returns a valid and optimal solution to the problem.
Proof: Consider an arbitrary instance of the problem.
Let S be the solution to this instance returned by our algorithm.
Let O be an optimal solution to the same instance.

What constitutes a solution?

Assume both S and O are organized so that they can be compared one choice at a time.

What order should the solutions be presented?

Assume that S and O disagree at some point and call this point i.

This step involves pseudo-induction.

We must argue that S and O agree on point i.

We can reach a contradiction under two possibilities:

1. The case contradicts how our algorithm works.
2. We can show that O is not valid or optimal in this case.

Otherwise we show how we can modify O so that it is still valid and optimal, but is also in agreement with S.

We have shown that S and O cannot disagree at point i or that we can modify O to agree with S at this point.
Conclude S is optimal. □

TCSS 343 – Design and Analysis of Algorithms

Museum Guards

Museum Guards

- The Museum of Glass needs help hiring guards for their next exhibit. In the exhibit there will be n pieces displayed in a long hallway. Each piece is placed at some point along the hallway. The Museum's insurance requires them to have a human guard within 5 feet of every piece in the museum. The Museum wishes to hire as few guards as possible.

- Can we formalize this problem?

Museum Guards
Input: $P[1 \dots n]$ a list of locations for n pieces of art
Output: $G[1 \dots k]$ a list of locations for k guards such that for each $P[i]$ there exists a $G[j]$ such that $|P[i] - G[j]| \leq 5$ and k is minimum

- What are the conditions of validity and optimality for this problem?
  - Validity: All pieces have a guard within five feet.
  - Optimality: We use the minimum number of guards.

- What is a good greedy strategy for placing the minimum number of guards to cover all the pieces of art?
  - Idea: Place a guard that covers the most pieces as possible.

Counter Examples

- Idea: Place a guard that covers the most pieces as possible.
  - This idea is, in fact, incorrect.
  - We can demonstrate this with a counter example.

- Finding a counter example means constructing an input for which the algorithm fails.
  - An algorithm fails an optimization problem if it gives a sub-optimal solution for at least one instance.
  - The more complex the algorithm is, the more complex the counter example usually must be.

- When thinking about counter examples we should think about aspects of the algorithm, and why they might lead us to error.
  - For this algorithm we place a guard to cover the most pieces of art as possible.
  - We should ask the question, “When will placing a guard on the most pieces possible cause us to use more guards than we need?”
  - The answer that will help us think if a counter example is, “When those pieces should (optimally) be guarded by two guards instead of one.”

Counter Examples

- Idea: Place a guard that covers the most pieces as possible.
  - We want to design an input that causes the algorithm to fail.
  - We want the input to have several pieces that the algorithm will guard with a single guard, but this is in error. The optimal solution will guard these same pieces with two guards instead of one.

- Finding a counter example is a bit of an art and usually progresses through trial-and-error.
  - I will start by placing some pieces that can be all guarded by a single guard.

    $$\{10, 11, 12\}$$
  - Since these can be optimally guarded by 1 guard our algorithm is optimal on this input. So, this is not a sufficient counter example. However, we can expand on it to reach a counter example.
  - I'm going to add another item that forces the optimal solution to use 2 guards.

    $$\{1,10,11,12\}$$
  - We can place a guard at position 6 and they will guard the items at positions $1, 10$, and 11 but not 12. This is optimal but is also a possible solution from our algorithm.
  - I want to force our algorithm to pick the items $10,11$, and 12 by adding another item 13.

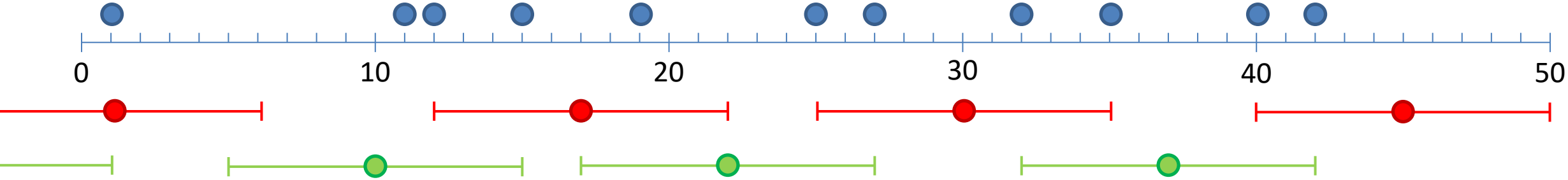
    $$\{1,10,11,12,13\}$$
  - Now our algorithm will place one guard to guard $\{10,11,12,13\}$ and another to guard $\{1\}$, but this is still optimal.
  - We can complete this counter example by forcing our algorithm to use a third guard when optimally we still need only two.

    $$\{1,10,11,12,13,22\}$$
  - This final counter example works since the optimal placement of guards is at positions $\{6,17\}$. Our algorithm will place one to cover the items $\{10,11,12,13\}$, and then will need another for the item at 1 and another for the piece 22. Thus, our algorithm will place three guards when only two are needed.

Museum Guards

- Idea: We know we must place a guard for the left-most piece of art. Where should we place them, so they guard as many pieces as possible in addition to this piece?
  - We should place it as far to the right as possible.

- Note: There is more than one optimal solution to this instance.



Proof of Correctness

Theorem: If we repeatedly place a guard five feet to the right of the left-most unguarded piece of art, we will minimize the number of guards placed.

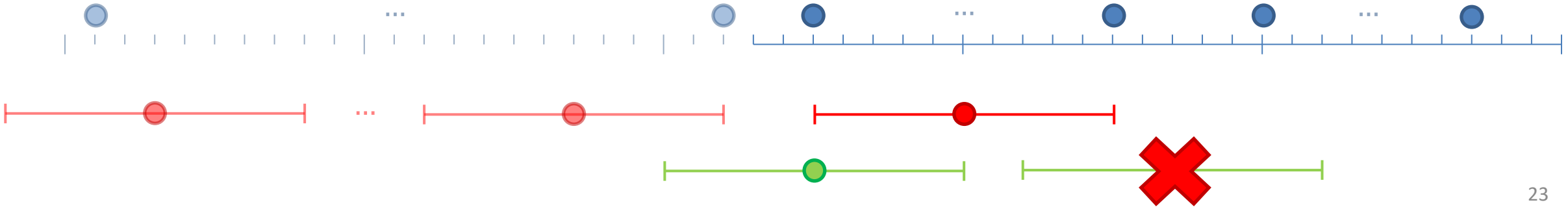
Proof: Consider the guard we will place to cover the left-most unguarded piece of art and label the location we place this guard g.

Imagine placing the guard at another location g'.

g' cannot be further to the right than g because that would leave the left-most piece unguarded. So g' must be to the left of g.

There are no unguarded pieces to the left of the left-most unguarded piece, so there are no previously unguarded pieces guarded by g' that are not also guarded by g.

So, a guard placed at position g will guard an equal number or more pieces than any other guard placed to guard the left-most unguarded item. □



Proof of Correctness

Theorem: If we repeatedly place a guard five feet to the right of the left-most unguarded piece of art, we will minimize the number of guards placed.

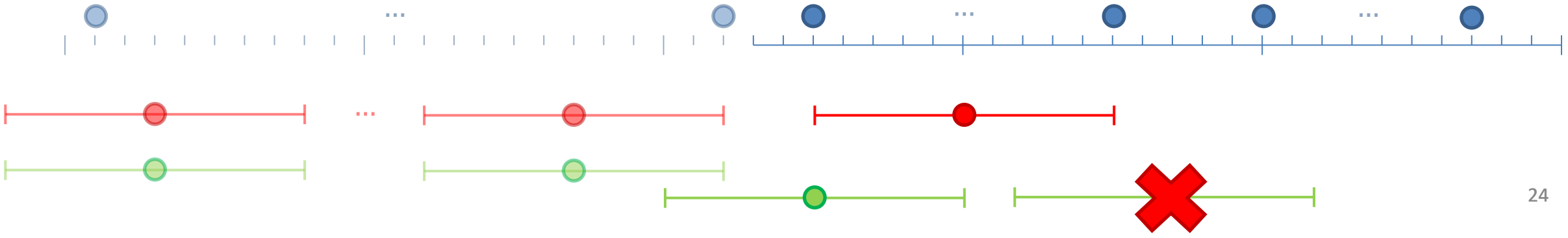
Proof: Assume we have an arbitrary set of pieces of art in order of the position in the exhibit from left-to-right.
Assume S are the positions our algorithm will place the guards from left-to-right.
Assume O are the positions of a collection of optimally placed guards from left-to-right.

Let i be the left-most guard such that $s_i \neq o_i$. At this point there is a left-most piece of art that is not guarded by the first $i-1$ guards. Call this piece a_i.

If o_i is to the right of s_i it leaves a_i unguarded and thus is invalid. Since we have assumed O is optimal we know this case can't occur.

So o_i is to the left of s_i. Now imagine replacing o_i in O with s_i to get O'. Since all pieces to the left of a_i are already guarded, this replacement does not leave any pieces unguarded, and thus remains valid. In addition, we have replaced a guard, so the total number of guards is still optimal. So, O' is an optimal solution that also places its ith guard at position s_i.

We can continue this proof with O' in place of O, and we can repeat this argument for all disagreements.
Thus, is sufficient to show that there exists an optimal solution that agrees with S on all guard placements, and so S is optimal. □



TCSS 343 – Design and Analysis of Algorithms

Non-Overlapping Intervals

Non-Overlapping Intervals

- You, Alice and Bob are attending a computer science conference at which there are numerous talks, panels, exhibits and other events you want to attend. They all start and end at different times and its impossible to attend everything due to overlapping time periods. Together you have selected the events you want to attend and have decided you want to attend as many events as possible to maximize your experience (but for any event you attend you will attend the entire event). Alice has recorded the start and end time of every event you wish to attend. You and Bob are trying to figure out how to select events so that you will maximize the number of events you can attend.

- Can we formalize this problem?

Conference Events
Input: $C = \{(s_i, f_i) | 1 \leq i \leq n\}$ the start and finish times for n events
Output: m a maximum number of events and $A \subseteq C$ such that $|A| = m$ and if $c_i \in A$ and $c_j \in A$ then either $f_i < s_j$ or $f_j < s_i$

- What are the conditions of validity and optimality for this problem?
  - A solution is valid if all events are non-overlapping.
  - A solution is optimal if it attends the maximum number of events.

- This problem is commonly known as the non-overlapping intervals problem.

Non-Overlapping Intervals

Non-Overlapping Intervals
Input: $C = \{(s_i, f_i) | 1 \leq i \leq n\}$ the start and finish times for n events
Output: m a maximum number of events and $A \subseteq C$ such that $|A| = m$ and if $c_i \in A$ and $c_j \in A$ then either $f_i < s_j$ or $f_j < s_i$

- Alice and Bob brainstormed a few greedy algorithms to solve this problem, but they don't know which ones (if any) are correct.
  - Always select the event with the earliest start time.
  - Always select the event with the earliest finish time.
  - Always select the event with the shortest duration.
  - Always select the event with the fewest number of conflicts.

- Which of these do you think are correct? Are any?

Finding Counter Examples

- When we come up with a greedy algorithm it is either correct or incorrect. Before trying to prove an algorithm correct it is worth trying to find a simple counterexample that shows it will not work.

- Always select the event with the earliest start time.



- Always select the event with the earliest finish time.
  - No counter example! It's correct!
- Always select the event with the shortest duration.



- Always select the event with the least conflicts.



Non-Overlapping Intervals

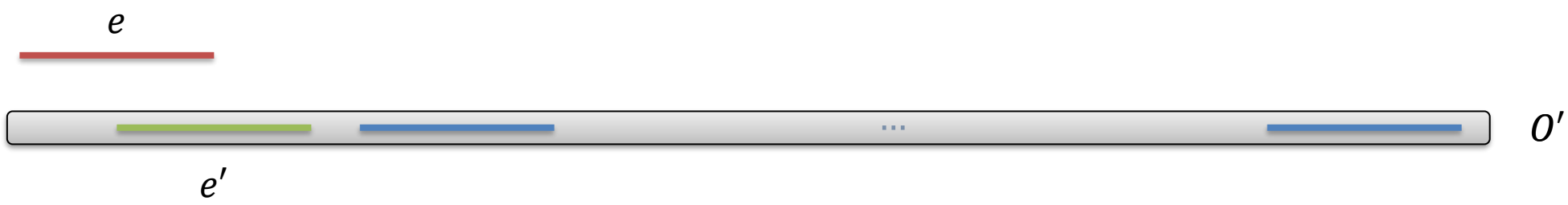
Theorem: Always selecting the non-overlapping event with the earliest finish time will result in a maximum set of non-overlapping events.

Proof: Consider the event with the earliest finish time, e, and consider any maximum sized set of non-overlapping events, O. If $e \in O$ we know selecting e is optimal.

Now consider that $e \notin O$ and that $e' \in O$ is the job with earliest finish time. Since e is the job with earliest finish time we know that e finishes earlier (or at the same time) as e'.

Every job in O other than e' starts after e' finishes, so they also start after e finishes.

So, we can remove e' from O and add in e to get O' and O' is still optimal. So e is part of an optimal set. □



Non-Overlapping Intervals

Theorem: Always selecting the non-overlapping event with the earliest finish time will result in a maximum set of non-overlapping events.

Proof: Assume we have an arbitrary set of n events with start and finish times.
Let $P = \{p_1, \dots, p_d\}$ be the set of non-overlapping events selected by our algorithm in ascending order of finish time.
Let $O = \{o_1, \dots, o_m\}$ be a maximum sized set of non-overlapping events in ascending order of finish time.
Let i be the first event such that P and O select different events, that is $p_i \neq o_i$. We know that p_i finishes no later than o_i as we always select the non-overlapping event with earliest finish time.

We will consider a solution O' constructed from O by removing o_i and adding p_i.
We know that p_i does not overlap with events $o_1, \dots, o_{i-1}$ since they are the same as $p_1, \dots, p_{i-1}$ and we only select non-overlapping events. Since the events $o_{i+1}, \dots, o_m$ do not overlap with o_i they must all have start times later than the finish time of o_i. Since p_i has a finish time no later than o_i these events will not overlap with p_i either. Thus, O' is valid because no events in the set overlap with one another.
O' is optimal because it has m non-overlapping events (we removed one and added one to O).
After the replacement O' is still optimal and now agrees with P at point i.

Repeating this argument for every event that disagrees produces an optimal solution that agrees with P on all event selections. Thus, P is optimal. □

